

UNIMA/AFYA

Welberth de Oliveira Costa
Igor Rafael Basilio de Lima
Danilo Vitor de Lima

GERENCIADOR DE TABELA DE SÍMBOLOS COM ESCOPOS ANINHADOS

Maceió/AL
2026.1

Welberth de Oliveira Costa
Igor Rafael Basilio de Lima
Danilo Vitor de Lima

GERENCIADOR DE TABELA DE SÍMBOLOS COM ESCOPOS ANINHADOS

Relatório técnico apresentado à disciplina de
Compiladores da UNIMA/AFYA, como requisito parcial
de avaliação. Orientador: Prof. Amaury Nogueira.

Maceió/AL
2026.1

SUMÁRIO

1 Introdução.....	4
2 Metodologia de Implementação	4
2.1 Organização modular.....	4
2.2 Estruturas de dados.....	4
2.3 Algoritmos principais	5
2.4 Trecho central do código	5
3 Casos de Teste	6
3.1 Entrada (chamadas executadas na demonstração).....	6
3.2 Saída (output real do terminal).....	7
3.3 Análise dos resultados	7
4 Considerações Finais	8

1 Introdução

A Tabela de Símbolos é uma das estruturas de dados centrais de um compilador. Ela é construída durante a análise léxica e sintática e consultada intensamente durante a análise semântica, sendo responsável por armazenar informações sobre os identificadores do programa, como nome, tipo e o escopo ao qual pertencem. Sem ela, o compilador não teria como verificar se uma variável foi declarada antes do uso, qual o seu tipo, ou se há conflito de nomes.

Programas reais possuem escopos aninhados: o corpo de uma função define um escopo interno ao escopo global, um laço define um escopo interno ao da função, e assim por diante. Uma variável declarada em um escopo interno pode “sombrear” (*shadowing*) outra de mesmo nome declarada em um escopo externo, e deve deixar de existir quando o escopo interno é fechado.

Este projeto implementa um gerenciador de Tabela de Símbolos que modela esse comportamento usando uma Pilha (Stack) de Hash Maps. O objetivo é expor os métodos declarar(variavel, tipo) e buscar(variavel), além das operações de abertura e fechamento de escopo, e demonstrar, passo a passo, a correta resolução de nomes entre escopos.

2 Metodologia de Implementação

A solução foi escrita em Python 3, sem dependências externas. A escolha das estruturas de dados segue diretamente a teoria de compiladores, conforme descrito a seguir.

2.1 Organização modular

O código foi dividido em quatro módulos com responsabilidades bem definidas, facilitando o trabalho em equipe: erros.py (erro semântico) e escopo.py (classe que representa um único escopo como Hash Map); tabela_de_simbolos.py (a Pilha de escopos e os métodos de declaração e busca); e main.py (demonstração com os casos de teste). O ponto de entrada é executado com o comando `python3 main.py`.

2.2 Estruturas de dados

Pilha (Stack): uma lista Python (`pilha_de_escopos`) é usada como pilha. A operação de empilhar (*push*) corresponde a `append()` e a de desempilhar (*pop*) a `pop()`. O topo da pilha é sempre o escopo mais interno (o escopo atual).

Hash Map (Hash Table): cada escopo é um dicionário Python, ou seja, uma tabela de espalhamento que mapeia o nome da variável (chave) ao seu tipo (valor). A consulta e a inserção em cada escopo ocorrem, em média, em tempo constante $O(1)$.

2.3 Algoritmos principais

declarar(variavel, tipo): insere o par no Hash Map do topo da pilha (escopo atual). Antes, verifica se a variável já existe naquele mesmo escopo; em caso afirmativo, lança um erro semântico de redeclaração. Declarar o mesmo nome em um escopo mais interno é permitido, isso é o *shadowing*.

buscar(variavel): percorre a pilha do topo (escopo interno) para a base (escopo global). Retorna o tipo no primeiro escopo em que a variável for encontrada, garantindo que a versão mais interna tenha prioridade. Se nenhum escopo a contiver, retorna None, indicando uso de variável não declarada.

entrar_escopo() / sair_escopo(): empilham um novo Hash Map vazio e desempilham o escopo do topo, respectivamente. Ao sair de um escopo, todas as variáveis locais nele declaradas deixam de ser acessíveis, o que reproduz fielmente o tempo de vida das variáveis locais.

2.4 Trecho central do código

```
class TabelaDeSimbolos:
    def __init__(self):
        self.pilha_de_escopos = []
        # Toda tabela começa com um escopo global aberto
        self.entrar_escopo(nome="global")

    def entrar_escopo(self, nome="anonimo"):
        # PUSH: empilha um novo Escopo (Hash Map) no topo da pilha
        self.pilha_de_escopos.append(Escopo(nome))
        print(f"[ESCOPO] Entrou no escopo '{nome}' "
              f"(profundidade = {self.profundidade()})")

    def declarar(self, variavel, tipo):
        # Declara a variável no escopo do TOPO da pilha (escopo atual)
        escopo_atual = self.pilha_de_escopos[-1]
        escopo_atual.declarar_local(variavel, tipo)
        print(f"[DECLARAR] '{variavel}' : {tipo} adicionada ao escopo "
              f"'{escopo_atual.nome}'")
        return True

    def buscar(self, variavel):
        # Busca a variável percorrendo a PILHA do TOPO para a BASE
        for escopo in reversed(self.pilha_de_escopos):
            if escopo.contem(variavel):
                tipo = escopo.buscar_local(variavel)
                print(f"[BUSCAR ] '{variavel}' ENCONTRADA no escopo "
                      f"'{escopo.nome}' -> tipo = {tipo}")
                return tipo
        print(f"[BUSCAR ] '{variavel}' NÃO ENCONTRADA (erro: não declarada)")
        return None
```

3 Casos de Teste

O programa executa automaticamente seis casos de teste que cobrem os comportamentos relevantes da Tabela de Símbolos. A seguir, a entrada (*input*, expressa como chamadas de método) e a saída (*output*) reais capturadas da execução de python3 main.py.

3.1 Entrada (chamadas executadas na demonstração)

```
tabela = TabelaDeSimbolos()

print("\n--- CASO 1: Declarações no escopo global ---")
tabela.declarar("x", "int")
tabela.declarar("nome", "string")
tabela.buscar("x")
tabela.buscar("nome")

print("\n--- CASO 2: Escopo aninhado (função) e acesso ao externo ---")
tabela.entrar_escopo(nome="funcao_soma")
tabela.declarar("a", "float")
tabela.declarar("b", "float")
tabela.buscar("a")
tabela.buscar("x")

print("\n--- CASO 3: Shadowing (sombreamento de variável) ---")
tabela.declarar("x", "double")
tabela.buscar("x")

print("\n--- CASO 4: Saída de escopo e perda das variáveis locais ---")
tabela.sair_escopo()
tabela.buscar("a")
tabela.buscar("x")

print("\n--- CASO 5: Erro de redeclaração no mesmo escopo ---")
try:
    tabela.declarar("x", "char")
except ErroSemantico as e:
    print(f"[ERRO ] {e}")

print("\n--- CASO 6: Variável inexistente ---")
resultado = tabela.buscar("inexistente")
print(f"Resultado da busca por 'inexistente': {resultado}")

print("\n" + "=" * 60)
print(" FIM DA SIMULAÇÃO")
print("\n" + "=" * 60)
```

3.2 Saída (output real do terminal)

```
=====
SIMULAÇÃO DE GERENCIADOR DE TABELA DE SÍMBOLOS
=====
[ESCOPO] Entrou no escopo 'global' (profundidade = 1)

--- CASO 1: Declarações no escopo global ---
[DECLARAR] 'x : int' adicionada ao escopo 'global'.
[DECLARAR] 'nome : string' adicionada ao escopo 'global'.
[BUSCAR ] 'x' ENCONTRADA no escopo 'global' -> tipo = int
[BUSCAR ] 'nome' ENCONTRADA no escopo 'global' -> tipo = string

--- CASO 2: Escopo aninhado (função) e acesso ao externo ---
[ESCOPO] Entrou no escopo 'funcao_soma' (profundidade = 2)
[DECLARAR] 'a : float' adicionada ao escopo 'funcao_soma'.
[DECLARAR] 'b : float' adicionada ao escopo 'funcao_soma'.
[BUSCAR ] 'a' ENCONTRADA no escopo 'funcao_soma' -> tipo = float
[BUSCAR ] 'x' ENCONTRADA no escopo 'global' -> tipo = int

--- CASO 3: Shadowing (sombreamento de variável) ---
[DECLARAR] 'x : double' adicionada ao escopo 'funcao_soma'.
[BUSCAR ] 'x' ENCONTRADA no escopo 'funcao_soma' -> tipo = double

--- CASO 4: Saída de escopo e perda das variáveis locais ---
[ESCOPO] Saiu do escopo 'funcao_soma' (profundidade = 1)
[BUSCAR ] 'a' NÃO ENCONTRADA (erro: não declarada)
[BUSCAR ] 'x' ENCONTRADA no escopo 'global' -> tipo = int

--- CASO 5: Erro de redeclaração no mesmo escopo ---
[ERRO  ] Variável 'x' já declarada no escopo atual 'global'.

--- CASO 6: Variável inexistente ---
[BUSCAR ] 'inexistente' NÃO ENCONTRADA (erro: não declarada)
Resultado da busca por 'inexistente': None

=====
FIM DA SIMULAÇÃO
=====
```

3.3 Análise dos resultados

Os resultados confirmam o comportamento esperado. No Caso 2, a busca por x dentro da função encontra a variável do escopo global, provando que escopos internos enxergam os externos. No Caso 3, após declarar x como double no escopo interno, a busca passa a retornar double em vez do int global, o *shadowing* funciona. No Caso 4, ao sair do escopo, a variável a desaparece e x volta a ser o int global. Os Casos 5 e 6 demonstram o tratamento de erros de redeclaração e de variável não declarada.

4 Considerações Finais

A implementação demonstrou, na prática, como uma Pilha de Hash Maps modela com fidelidade os escopos aninhados de uma linguagem de programação. A separação entre a classe que representa um escopo individual e a classe que gerencia a pilha tornou o código modular e de fácil manutenção, além de evidenciar de forma clara os conceitos de resolução de nomes, sombreamento e tempo de vida das variáveis estudados na disciplina.

REFERÊNCIAS

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores:** princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison Wesley, 2008.

LOUDEN, Kenneth C. **Compiladores:** princípios e práticas. São Paulo: Cengage Learning, 2004.

PYTHON SOFTWARE FOUNDATION. **Python 3 documentation.** Disponível em: <https://docs.python.org/3/>. Acesso em: 9 jun. 2026.